

AADK 10: Task 1 — Design Your Own Data Model and Blueprint Class

Student Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – Session 10

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

1. Concept Review & Notes

What is a class in Kotlin?

A **class in Kotlin** is a blueprint used to create objects. It defines the **properties (data)** and **functions (behavior)** that the objects created from the class will have. Classes help organize code and represent real-world entities in programs.

Difference between a class and an object

Class	Object
Blueprint or template	Instance of the class
Defines properties and functions	Contains actual values
Does not hold real data	Holds real data

Example:

- **Class:** Recipe
- **Object:** Recipe("Pasta", listOf("Noodles","Sauce"), 20)

What are primary and secondary constructors?

Primary Constructor

- Defined in the class header.
- Used to initialize class properties quickly.

Example:

```
class Student(val name: String, val age: Int)
```

Secondary Constructor

- Defined inside the class body.
- Used when additional initialization logic is required.

Example:

```
class Student {  
    var name: String  
    var age: Int  
  
    constructor(name: String, age: Int){  
        this.name = name  
        this.age = age  
    }  
}
```

Purpose of init blocks and member functions

init block

- Runs automatically when an object is created.
- Used for validation or initialization logic.

Example:

```
init {  
    println("Student created")  
}
```

Member functions

- Functions inside a class.
- Used to perform actions related to that class.

Example:

```
fun displayName(){  
    println(name)  
}
```

Example of a Simple Kotlin Data Class

```
data class Contact(  
    val name: String,  
    val phone: String,  
    val email: String  
)
```

This class automatically provides:

- toString()
 - equals()
 - copy()
 - hashCode()
-

2. Design Your Model

Domain Chosen

Student Tracker App

Class Design

```
class Student(  
    val name: String,  
    val course: String,
```

```

    val marks: Int,
    var isPassed: Boolean,
    val subjects: List<String>
) {

    init {
        println("Student record created for $name")
    }

    fun calculateGrade(): String {
        return if (marks >= 50) "Pass" else "Fail"
    }

    fun toggleStatus() {
        isPassed = !isPassed
    }
}

```

Properties

- name: String
- course: String
- marks: Int
- isPassed: Boolean
- subjects: List<String>

Functions

- calculateGrade() – determines pass or fail
 - toggleStatus() – toggles pass status
-

3. UML Style Diagram

```
-----  
|           Student           |  
-----  
| name : String               |  
| course : String             |  
| marks : Int                 |  
| isPassed : Boolean          |  
| subjects : List<String>     |  
-----  
| calculateGrade() : String   |  
| toggleStatus() : Unit      |  
-----
```

Object Instances

```
val student1 = Student(  
    "Rajat",  
    "Computer Science",  
    78,  
    true,  
    listOf("Kotlin", "Android", "Database")  
)
```

```
val student2 = Student(  
    "Aman",  
    "Computer Science",  
    42,  
    false,  
    listOf("Java", "Networks", "OS")  
)
```

)

Function Calls Example

```
println(student1.calculateGrade())
```

```
println(student2.calculateGrade())
```

```
student2.toggleStatus()
```

```
println(student2.isPassed)
```

4. Compose Integration Plan

In **Jetpack Compose**, these student objects could be displayed in a **LazyColumn list**. Each student item could show the name, course, and marks. A **button or icon** could allow the user to toggle the pass status. The UI would update automatically using **state management** when the student data changes.

5. Reflection

Designing a class helped me understand how to organize data and logic into a reusable structure. I learned how properties represent data while functions define behaviors. Using init blocks allows validation during object creation. Modeling data early makes UI development easier because the UI can directly use structured data objects. This approach also keeps code cleaner, easier to maintain, and more scalable.